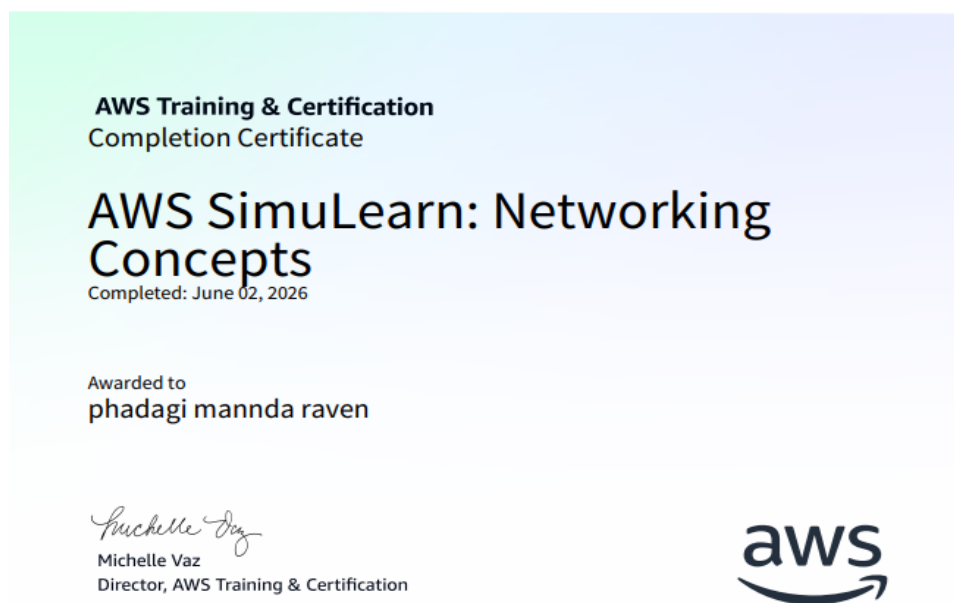


AWS SimuLearn: Networking Concepts

Complete Hands-On Lab Documentation



Completed: June 02, 2026

Awarded to: Phadagi Mannda Raven

Issued by: AWS Training & Certification

This document serves as comprehensive documentation of the AWS SimuLearn: Networking Concepts lab completion, including step-by-step explanations, key concepts, and practical implementation details.

Table of Contents

- 1. Introduction & Overview
- 2. Lab Prerequisites
- 3. Core Networking Concepts Covered
- 4. Step-by-Step Lab Walkthrough
- 5. Hands-On Exercises
- 6. Key Takeaways & Best Practices
- 7. Troubleshooting Guide
- 8. Conclusion & Next Steps

1. Introduction & Overview

The **AWS SimuLearn: Networking Concepts** lab is an interactive, hands-on training module designed by AWS Training & Certification to help learners understand fundamental networking concepts within the Amazon Web Services (AWS) ecosystem. This lab provides a simulated environment where participants can practice configuring and managing AWS networking resources without incurring real cloud costs or risking production environments.

Learning Objectives:

- Understand AWS Virtual Private Cloud (VPC) architecture and components
- Learn to configure subnets, route tables, and internet gateways
- Implement security groups and network ACLs for traffic control
- Understand IP addressing, CIDR blocks, and subnetting in AWS
- Practice connecting resources across different network segments
- Learn fundamental troubleshooting techniques for network connectivity issues

Lab Environment Architecture

The lab simulates a typical AWS environment with the following components:

VPC (Virtual Private Cloud): A logically isolated network within the AWS cloud where you can launch AWS resources.

Subnets: Segments of a VPC's IP address range where you can place groups of isolated resources.

Internet Gateway (IGW): A horizontally scaled, redundant, and highly available VPC component that allows communication between instances in your VPC and the internet.

Route Tables: A set of rules, called routes, that are used to determine where network traffic is directed.

Security Groups: Virtual firewalls that control inbound and outbound traffic for EC2 instances.

Network ACLs: Optional layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets.

2. Lab Prerequisites

Before starting this lab, ensure you have the following:

Knowledge Prerequisites:

- Basic understanding of computer networking concepts (IP addresses, subnets, routing)
- Familiarity with TCP/IP protocol stack
- Understanding of basic security concepts (firewalls, access control)
- General cloud computing awareness

Technical Requirements:

- An AWS Training account or AWS Skill Builder subscription
- Modern web browser (Chrome, Firefox, Safari, or Edge)
- Stable internet connection
- No additional software installation required (browser-based simulation)

Time Commitment:

- Estimated completion time: 45-60 minutes
- Self-paced learning with interactive checkpoints

3. Core Networking Concepts Covered

3.1 Virtual Private Cloud (VPC)

A **VPC** is a virtual network dedicated to your AWS account. It is logically isolated from other virtual networks in the AWS Cloud.

Key Characteristics:

- You have complete control over your virtual networking environment
- You can select your own IP address range, create subnets, and configure route tables and network gateways
- You can use both IPv4 and IPv6 in your VPC for secure and easy access to resources and applications
- Default VPCs are automatically created for each AWS region in your account

CIDR Blocks:

When creating a VPC, you specify an IPv4 CIDR block (e.g., 10.0.0.0/16). The CIDR block size can range from /16 to /28. The first four IP addresses and the last IP address in each subnet CIDR block are reserved by AWS for internal networking purposes.

3.2 Subnets and IP Addressing

Subnets are partitions of a VPC's IP address range. Each subnet must reside entirely within one Availability Zone.

Subnet Types:

- **Public Subnets:** Subnets that have a route to an Internet Gateway. Resources in public subnets can communicate directly with the internet.
- **Private Subnets:** Subnets that do not have a route to an Internet Gateway. Resources in private subnets cannot be reached directly from the internet.
- **VPN-Only Subnets:** Subnets that have a route to a Virtual Private Gateway but not to an Internet Gateway.

IP Addressing in AWS:

- AWS reserves 5 IP addresses in each subnet (first 4 and last 1 in the CIDR range)
- For a subnet with CIDR 10.0.1.0/24, reserved addresses are: 10.0.1.0 (network), 10.0.1.1 (VPC router), 10.0.1.2 (Amazon DNS), 10.0.1.3 (future use), 10.0.1.255 (network broadcast)
- Available IPs for assignment: 10.0.1.4 through 10.0.1.254 (251 addresses)

3.3 Routing and Internet Gateways

Route Tables contain a set of rules (routes) that control traffic from your subnet or gateway.

Main Route Table:

- Automatically created when you create a VPC
- Controls routing for all subnets not explicitly associated with a custom route table
- Best practice: Leave the main route table in its default state and create custom route tables for specific routing needs

Internet Gateway (IGW):

- Horizontally scaled, redundant, and highly available VPC component
- Provides a target in your VPC route tables for internet-routable traffic
- Performs network address translation (NAT) for instances with public IPv4 addresses
- Must be attached to your VPC before resources can access the internet
- You can only attach one IGW to a VPC at a time

Default Route (0.0.0.0/0):

The most specific route that matches the traffic determines the traffic route. The 0.0.0.0/0 route is the least specific route and is used as a default route for internet-bound traffic when more specific routes don't match.

3.4 Security Groups and Network ACLs

AWS provides two levels of security for your VPC: **Security Groups** (instance level) and **Network ACLs** (subnet level).

Security Groups:

- Act as virtual firewalls for your EC2 instances
- Control inbound and outbound traffic at the instance level
- **Stateful:** If traffic is allowed in, the return traffic is automatically allowed, regardless of outbound rules
- You can specify allow rules only (cannot specify deny rules)
- All rules are evaluated before deciding whether to allow traffic
- By default, all inbound traffic is denied and all outbound traffic is allowed
- You can attach multiple security groups to an instance

Network ACLs (NACLs):

- Act as firewalls for associated subnets
- Control inbound and outbound traffic at the subnet level
- **Stateless:** Return traffic must be explicitly allowed by rules
- You can specify both allow and deny rules
- Rules are evaluated in numerical order (lowest to highest)
- The first matching rule is applied

- Default NACL allows all inbound and outbound traffic by default
- Each subnet must be associated with a NACL (if not explicitly associated, it uses the default NACL)

4. Step-by-Step Lab Walkthrough

This section provides a detailed walkthrough of the AWS SimuLearn: Networking Concepts lab. The lab is divided into multiple interactive modules, each building upon the previous one to create a complete understanding of AWS networking.

Module 1: Creating Your VPC

Objective: Create a custom VPC with appropriate CIDR block and basic configuration.

Step 1: Access the VPC Dashboard

- Navigate to the AWS Management Console (simulated environment)
- Select "VPC" from the Services menu or search for VPC in the search bar
- Click "Your VPCs" in the left navigation panel

Step 2: Create a New VPC

- Click the "Create VPC" button
- Choose "VPC and more" (automated) or "VPC only" (manual) option
- For this lab, we use the manual approach to understand each component
- Enter the following details:
 - **Name tag:** MyLabVPC
 - **IPv4 CIDR block:** 10.0.0.0/16 (provides 65,536 IP addresses)
 - **IPv6 CIDR block:** No IPv6 CIDR block (for simplicity)
 - **Tenancy:** Default (shared hardware)

Step 3: Verify VPC Creation

- After creation, the VPC appears in your VPC list
- Note the VPC ID (e.g., vpc-0a1b2c3d4e5f67890)
- The VPC is automatically assigned a main route table and default network ACL
- The default security group is also created automatically

Key Learning Point:

The CIDR block 10.0.0.0/16 is a private IP range (RFC 1918) that is not routable on the public internet, making it ideal for internal cloud networks. The /16 prefix means the first 16 bits are fixed, leaving 16 bits for host addresses ($2^{16} = 65,536$ possible addresses).

Module 2: Creating Subnets

Objective: Create public and private subnets within your VPC to segment resources.

Step 1: Navigate to Subnets

- In the VPC dashboard, click "Subnets" in the left navigation panel
- Click "Create subnet"
- Select your newly created VPC (MyLabVPC)

Step 2: Create Public Subnet

- Click "Add new subnet"
- Configure the public subnet:
 - **Subnet name:** PublicSubnet-1A
 - **Availability Zone:** us-east-1a (or equivalent in your region)
 - **IPv4 CIDR block:** 10.0.1.0/24 (256 addresses, 251 usable)
- Click "Create subnet"

Step 3: Create Private Subnet

- Click "Add new subnet" again
- Configure the private subnet:
 - **Subnet name:** PrivateSubnet-1A
 - **Availability Zone:** us-east-1a (same AZ for demonstration)
 - **IPv4 CIDR block:** 10.0.2.0/24 (256 addresses, 251 usable)
- Click "Create subnet"

Step 4: Verify Subnet Configuration

- Both subnets should appear in the subnet list
- Verify they are associated with the correct VPC
- Note that both subnets are currently using the main route table
- Check that the CIDR blocks do not overlap and are within the VPC range (10.0.0.0/16)

Key Learning Point:

Subnetting allows you to organize and secure resources. Public subnets host resources that need internet access (web servers, load balancers), while private subnets host sensitive resources (databases, application servers) that should not be directly accessible from the internet.

Module 3: Configuring Internet Gateway

Objective: Create and attach an Internet Gateway to enable internet connectivity for public subnets.

Step 1: Create Internet Gateway

- In the VPC dashboard, click "Internet gateways" in the left navigation
- Click "Create internet gateway"
- Enter name tag: MyLabIGW

- Click "Create internet gateway"
- The IGW is created in a detached state

Step 2: Attach Internet Gateway to VPC

- Select the newly created IGW
- Click "Actions" → "Attach to VPC"
- Select your VPC (MyLabVPC)
- Click "Attach internet gateway"
- Verify the attachment state shows "Attached"

Step 3: Update Route Table for Public Subnet

- Navigate to "Route tables" in the left panel
- Select the main route table associated with your VPC
- Click "Actions" → "Edit routes"
- Add a new route:
 - **Destination:** 0.0.0.0/0 (all internet traffic)
 - **Target:** Internet Gateway → select MyLabIGW
- Click "Save changes"

Important Note:

In production environments, you should create a custom route table for the public subnet rather than modifying the main route table. This prevents accidental internet exposure of private subnets that use the main route table by default.

Key Learning Point:

The Internet Gateway is a redundant, highly available component that provides internet access. The route table entry 0.0.0.0/0 → IGW tells AWS to send all traffic not matching more specific routes to the internet gateway. Without this route, instances in the subnet cannot reach the internet.

Module 4: Configuring Security Groups

Objective: Create and configure security groups to control traffic to your EC2 instances.

Step 1: Navigate to Security Groups

- In the VPC dashboard, click "Security groups" in the left navigation
- Click "Create security group"

Step 2: Create Web Server Security Group

- Configure the security group:
 - **Security group name:** WebServer-SG

- **Description:** Security group for web servers
- **VPC:** MyLabVPC
- Configure inbound rules:
 - **Rule 1:** Type: HTTP, Protocol: TCP, Port range: 80, Source: 0.0.0.0/0
 - **Rule 2:** Type: HTTPS, Protocol: TCP, Port range: 443, Source: 0.0.0.0/0
 - **Rule 3:** Type: SSH, Protocol: TCP, Port range: 22, Source: My IP (or specific IP)
- Configure outbound rules (leave default: All traffic allowed)
- Click "Create security group"

Step 3: Create Database Security Group

- Create another security group:
 - **Security group name:** Database-SG
 - **Description:** Security group for database servers
 - **VPC:** MyLabVPC
- Configure inbound rules:
 - **Rule 1:** Type: MySQL/Aurora, Protocol: TCP, Port range: 3306, Source: WebServer-SG (not 0.0.0.0/0!)
- Click "Create security group"

Key Learning Point:

Security groups are **stateful** - if you allow inbound traffic, the corresponding outbound response is automatically allowed. You can reference other security groups as sources (e.g., Database-SG allowing traffic from WebServer-SG), which creates a dynamic security boundary that adjusts as instances are added or removed from the referenced security group.

Module 5: Configuring Network ACLs

Objective: Implement Network ACLs as an additional layer of security at the subnet level.

Step 1: Navigate to Network ACLs

- In the VPC dashboard, click "Network ACLs" in the left navigation
- The default NACL for your VPC is already created
- Click "Create network ACL" to create a custom one

Step 2: Create Custom NACL for Public Subnet

- Configure the NACL:
 - **Name tag:** PublicSubnet-NACL
 - **VPC:** MyLabVPC
- Click "Create network ACL"

Step 3: Configure Inbound Rules

- Select your new NACL and click "Inbound rules" tab
- Click "Edit inbound rules"
- Add rules in order (rules are evaluated numerically):
 - **Rule 100:** Type: HTTP, Protocol: TCP, Port: 80, Source: 0.0.0.0/0, Allow
 - **Rule 110:** Type: HTTPS, Protocol: TCP, Port: 443, Source: 0.0.0.0/0, Allow
 - **Rule 120:** Type: SSH, Protocol: TCP, Port: 22, Source: 10.0.0.0/16, Allow
 - **Rule 200:** Type: All traffic, Protocol: All, Port: All, Source: 0.0.0.0/0, Deny (explicit deny)
- Click "Save changes"

Step 4: Configure Outbound Rules

- Click "Outbound rules" tab
- Click "Edit outbound rules"
- Add rules:
 - **Rule 100:** Type: HTTP, Protocol: TCP, Port: 80, Destination: 0.0.0.0/0, Allow
 - **Rule 110:** Type: HTTPS, Protocol: TCP, Port: 443, Destination: 0.0.0.0/0, Allow
 - **Rule 120:** Type: Custom TCP, Protocol: TCP, Port: 1024-65535, Destination: 0.0.0.0/0, Allow (ephemeral ports for return traffic)
 - **Rule 200:** Type: All traffic, Protocol: All, Port: All, Destination: 0.0.0.0/0, Deny
- Click "Save changes"

Step 5: Associate with Subnet

- Click "Subnet associations" tab
- Click "Edit subnet associations"
- Select PublicSubnet-1A
- Click "Save changes"

Key Learning Point:

NACLs are **stateless**, meaning you must explicitly allow both inbound and outbound traffic. The ephemeral port range (1024-65535) must be allowed in outbound rules for return traffic from internet requests. NACLs provide a subnet-level firewall that acts as a secondary defense if security group rules are misconfigured.

5. Hands-On Exercises

The following exercises reinforce the concepts learned in the lab. Complete these in the AWS SimuLearn environment or in your own AWS account (note: real AWS usage may incur costs).

Exercise 1: Launch EC2 Instances in Different Subnets

Scenario: Launch a web server in the public subnet and a database server in the private subnet.

Tasks:

1. Launch an EC2 instance in PublicSubnet-1A:

- AMI: Amazon Linux 2
- Instance type: t2.micro (free tier eligible)
- Network: MyLabVPC
- Subnet: PublicSubnet-1A
- Auto-assign public IP: Enable
- Security group: WebServer-SG
- Add user data script to install Apache web server

2. Launch an EC2 instance in PrivateSubnet-1A:

- AMI: Amazon Linux 2
- Instance type: t2.micro
- Network: MyLabVPC
- Subnet: PrivateSubnet-1A
- Auto-assign public IP: Disable
- Security group: Database-SG

3. Verify connectivity:

- Access the web server via its public IP (should display Apache test page)
- Attempt to SSH directly to the database server (should fail - no public IP)
- SSH to the web server, then SSH from there to the database server using private IP

Expected Outcome:

The web server is accessible from the internet, while the database server is only accessible from within the VPC. This demonstrates the principle of defense in depth and proper network segmentation.

Exercise 2: Testing Connectivity and Troubleshooting

Scenario: Troubleshoot connectivity issues between instances in different subnets.

Tasks:

1. Test VPC Internal Connectivity:

- From the web server, ping the database server using its private IP
- Command: ping 10.0.2.x
- Expected: Success (same VPC, no firewall blocking ICMP)

2. Test Internet Connectivity from Public Subnet:

- From the web server, ping a public IP (e.g., 8.8.8.8)
- Command: ping 8.8.8.8
- Expected: Success (public subnet has IGW route)

3. Test Internet Connectivity from Private Subnet:

- From the database server (via bastion host), ping 8.8.8.8
- Expected: Failure (private subnet has no IGW route)

4. Verify Security Group Rules:

- Attempt to access the web server on port 80 from your local machine
- Expected: Success (WebServer-SG allows HTTP)
- Attempt to access the web server on port 3306
- Expected: Failure (no rule allowing MySQL traffic to web server)

5. Verify NACL Rules:

- Temporarily modify the NACL to deny HTTP traffic
- Verify the web server becomes inaccessible
- Restore the rule and verify access returns

Troubleshooting Commands:

- ping [IP] - Test basic connectivity
- traceroute [IP] - Trace the network path
- curl -v [URL] - Test HTTP/HTTPS connectivity
- netstat -tuln - Check listening ports
- iptables -L - Check local firewall rules (on instance)
- AWS VPC Flow Logs - Analyze network traffic at the VPC level

Exercise 3: Implementing a Bastion Host Architecture

Scenario: Create a secure bastion host (jump box) for administrative access to private instances.

Tasks:

1. Create a dedicated security group for the bastion host:

- Name: Bastion-SG
- Inbound: SSH (22) from your specific IP only (NOT 0.0.0.0/0)
- Outbound: SSH (22) to the VPC CIDR (10.0.0.0/16)

2. Launch a bastion host in PublicSubnet-1A:

- Use Amazon Linux 2 AMI
- Assign the Bastion-SG
- Use a key pair for SSH access

3. Update Database-SG:

- Allow SSH (22) from Bastion-SG (not from 0.0.0.0/0)
- Remove any overly permissive SSH rules

4. Test the architecture:

- SSH to bastion host: `ssh -i key.pem ec2-user@bastion-public-ip`
- From bastion, SSH to database: `ssh -i key.pem ec2-user@db-private-ip`
- Verify direct SSH to database from internet fails

Security Best Practices:

- Never allow SSH from 0.0.0.0/0 in production
- Use specific IP ranges or VPN connections for administrative access
- Consider using AWS Systems Manager Session Manager instead of bastion hosts
- Implement multi-factor authentication (MFA) for bastion access
- Monitor and log all bastion host connections

6. Key Takeaways & Best Practices

6.1 VPC Design Principles

- Always plan your IP addressing scheme before creating VPCs
- Use private IP ranges (RFC 1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
- Leave room for growth - don't use the smallest possible CIDR blocks
- Document your network architecture with diagrams
- Use multiple Availability Zones for high availability

6.2 Subnet Design Best Practices

- Use public subnets only for resources that require direct internet access
- Place databases, application servers, and internal services in private subnets
- Use separate subnets for different tiers (web, application, database)
- Consider using /24 subnets for most use cases (251 usable IPs)
- Reserve some CIDR space for future expansion

6.3 Security Configuration

- Follow the principle of least privilege - allow only necessary traffic
- Use security groups as your primary defense (instance level)
- Use NACLs as a secondary defense (subnet level)
- Regularly audit and review security group rules
- Use security group references instead of IP addresses when possible
- Enable VPC Flow Logs for traffic analysis and troubleshooting

6.4 Routing Best Practices

- Create custom route tables instead of modifying the main route table
- Explicitly associate subnets with appropriate route tables
- Use specific routes instead of broad 0.0.0.0/0 routes when possible
- Document the purpose of each route table and route
- Regularly review routes for unnecessary internet access

6.5 Common Mistakes to Avoid

- Placing sensitive resources in public subnets
- Using 0.0.0.0/0 as source in security group rules (except for public web services)
- Forgetting to allow return traffic in NACLs (ephemeral ports)
- Overlapping CIDR blocks between VPCs or on-premises networks
- Not monitoring network traffic with VPC Flow Logs
- Using the default security group without customization
- Not implementing multi-AZ architecture for production workloads

7. Troubleshooting Guide

7.1 Instance Cannot Reach the Internet

Symptoms: ping 8.8.8.8 fails, cannot download updates

Checklist:

1. Verify the instance is in a public subnet (has route to IGW)
2. Check that the subnet's route table has 0.0.0.0/0 → IGW route
3. Confirm the instance has a public IP or Elastic IP
4. Verify security group allows outbound traffic
5. Check NACL allows outbound traffic and return traffic (ephemeral ports)
6. Verify the IGW is attached to the correct VPC

7.2 Cannot SSH to Instance

Symptoms: SSH connection times out or is refused

Checklist:

1. Verify security group allows SSH (22) from your IP
2. Check NACL allows inbound SSH and outbound ephemeral ports
3. Confirm the instance is in a public subnet with IGW route
4. Verify the instance has a public IP assigned
5. Check that the correct key pair is being used
6. Ensure the instance is running and passed status checks
7. Verify SSH service is running on the instance

7.3 Instances in Same VPC Cannot Communicate

Symptoms: Cannot ping or connect to other instances in the same VPC

Checklist:

1. Verify both instances are in the same VPC
2. Check security groups allow traffic between instances
3. Verify NACLs allow the required traffic
4. Confirm instances are in subnets with proper route table associations
5. Check if source/destination check is disabled (for NAT instances)
6. Verify VPC Flow Logs for traffic analysis

7.4 VPC Flow Logs Analysis

VPC Flow Logs capture information about IP traffic going to and from network interfaces in your VPC.

Common Flow Log Fields:

- **srcaddr/dstaddr:** Source and destination IP addresses
- **srcport/dstport:** Source and destination ports
- **protocol:** IANA protocol number (6=TCP, 17=UDP, 1=ICMP)
- **action:** ACCEPT or REJECT (based on security group/NACL rules)
- **log-status:** OK, NODATA, or SKIPDATA

Interpreting Flow Logs:

- **ACCEPT:** Traffic was allowed by security groups/NACLs
- **REJECT:** Traffic was denied - check your security group and NACL rules
- Look for patterns in rejected traffic to identify misconfigurations

8. Conclusion & Next Steps

8.1 Lab Completion Summary

Congratulations on completing the **AWS SimuLearn: Networking Concepts** lab! You have successfully demonstrated understanding of fundamental AWS networking concepts including VPC creation, subnet configuration, routing, security groups, and network ACLs.

Skills Acquired:

- Designing and implementing VPC architectures
- Configuring public and private subnets for secure resource placement
- Implementing Internet Gateways for controlled internet access
- Creating and managing security groups for instance-level security
- Configuring Network ACLs for subnet-level security
- Troubleshooting network connectivity issues
- Understanding the relationship between different networking components

8.2 Recommended Next Steps

To continue your AWS networking journey, consider the following resources and certifications:

Advanced Labs:

- AWS SimuLearn: Advanced VPC Networking
- AWS SimuLearn: Hybrid Connectivity (VPN and Direct Connect)
- AWS SimuLearn: Network Security and Compliance

AWS Certifications:

- **AWS Certified Cloud Practitioner** - Foundational understanding of AWS services
- **AWS Certified Solutions Architect – Associate** - Design and implementation of AWS architectures
- **AWS Certified Advanced Networking – Specialty** - Deep networking expertise in AWS

Additional Resources:

- AWS VPC Documentation: <https://docs.aws.amazon.com/vpc/>
- AWS Networking Best Practices Whitepaper
- AWS Well-Architected Framework - Security Pillar
- AWS re:Invent networking sessions (available on YouTube)

8.3 GitHub Repository Integration

This lab documentation is designed to be included in your GitHub repository as a comprehensive reference. Consider adding the following to your repo:

- This PDF documentation
- Architecture diagrams (draw.io or similar)
- Terraform/CloudFormation templates for automated VPC deployment
- Additional notes and lessons learned

- Links to your AWS certification achievements

8.4 Final Notes

Networking is the foundation of cloud architecture. Mastering AWS networking concepts is essential for building secure, scalable, and highly available applications. Continue practicing with real AWS environments (using free tier where possible) and stay updated with new AWS networking features and services.

Document Version: 1.0

Last Updated: June 02, 2026

Author: Phadagi Mannda Raven

Lab: AWS SimuLearn: Networking Concepts

Certificate Date: June 02, 2026